LOGITECH
EXPRESS





01 O PROBLEMA

A LOGITECH EXPRESS ENFRENTA UMA SÉRIE DE PROBLEMAS NO SISTEMA ATUAL:

- Motoristas cadastrados em duplicidade.
- Itens e ferramentas armazenados em textos separados por vírgulas.
- Dados sem normalização.
- Rotas, lógica de negócio e conexão com o banco misturadas.
- Requisições despadronizadas.
- Credenciais e tokens expostos diretamente no código.
- Servidor sujeito a travamentos e sobrecarga.



02

NOSSA SOLUÇÃO



MODELAGEM DE DADOS

Normalizar o banco e separar corretamente as entidades.



ARQUITETURA DO BACK-END

Utilizar MVC + Repository Pattern para separar responsabilidades.



SEGURANÇA E CONTRATOS

Utilizar .env + .gitignore e padronizar as requisições e respostas em JSON.

Essa estrutura prepara o sistema para a próxima Unidade Curricular de testes

03

BANCO DE DADOS NORMALIZADO

Principais entidades propostas:

- Motoristas
- Frotas
- Veículos
- Entregas
- Coletas
- Itens
- Frota_Itens

Objetivo da estrutura:

Cada informação fica armazenada em sua própria tabela, evitando duplicidade e dados desorganizados. Os relacionamentos serão feitos utilizando Chaves Primárias (PK) e Chaves Estrangeiras (FK).

Exemplo:

Um motorista pode realizar várias entregas, enquanto cada entrega pertence a um motorista.

05 RELACIONAMENTOS DO BANCO

MOTORISTA → ENTREGAS

1 : N
Um motorista pode realizar várias entregas.

FROTA → VEÍCULOS

1 : N
Uma frota pode possuir vários veículos.

FROTA ↔ ITENS

N : N
Uma frota pode possuir vários itens e um item pode estar relacionado a várias frotas.
Para resolver o relacionamento N, utilizamos a tabela intermediária:
Frota_Itens
A tabela guarda os IDs das duas entidades.

06

NORMALIZAÇÃO

O banco será organizado seguindo as regras de normalização:

1FN — ATOMICIDADE

Não teremos vários itens dentro de uma única célula.
Errado:
Espada, Escudo, Poção
Correto:
Cada item será armazenado separadamente.

2FN — DEPENDÊNCIA TOTAL

Cada informação ficará na tabela à qual realmente pertence.

3FN — SEM DEPENDÊNCIAS INDIRETAS

Informações que pertencem a outra entidade serão separadas.
Resultado:
Menos duplicidade, mais integridade e melhor organização dos dados.



07 ARQUITETURA DO BACK-END

- Utilizaremos uma arquitetura baseada em MVC + Repository Pattern.
- Model :Representa os dados e as entidades do sistema.
- Controller:Recebe as requisições e controla o fluxo da aplicação.
- Repository:Responsável pelo acesso ao banco de dados e pelas queries SQL.
- Routes:Define os endereços da API e encaminha as requisições para os Controllers.
- Objetivo: Separar cada responsabilidade e evitar que todo o código fique em um único arquivo.

08

ESTRUTURA DE PASTAS

```
src/
├controllers/
├models/
├repositores/
├routes/
└config/
```

Função das principais pastas:

- controllers/ → lógica das requisições
- models/ → entidades e estruturas dos dados
- repositories/ → comunicação com o banco
- routes/ → endpoints da API
- config/ → configurações da aplicação

ROTAS DA API



Motoristas

GET /motoristas
POST /motoristas
PUT /motoristas/:id
DELETE /motoristas/:id

Entregas

GET /entregas
POST /entregas
PUT /entregas/:id
DELETE /entregas/:id
Frotas

Frotas

GET /frotas
POST /frotas
PUT /frotas/:id
DELETE /frotas/:id



10 SEGURANÇA

As informações confidenciais não ficarão diretamente no código.
Utilizaremos um arquivo .env para armazenar:

DB_HOST=
DB_PORT=
DB_USER=
DB_PASSWORD=
DB_NAME=
API_TOKEN=

Também será utilizado o arquivo: .gitignore
para impedir que o .env seja enviado ao GitHub.

Objetivo: proteger senhas, chaves de acesso e tokens da aplicação.

11

CONTRATO JSON

A API utilizará um padrão de comunicação em JSON.

Exemplo de cadastro de motorista:

```
{
"nome": "Carlos Silva",
"cnh": "123456789",
"telefone": "47999999999"
}
```

Exemplo de resposta:

```
{
"id": 1,
"nome": "Carlos Silva",
"status": "cadastrado"
}
```

12 PLANO DE TESTES

Na próxima Unidade Curricular serão implementados testes para verificar:

- Cadastro de motorista.
- Atualização de motorista.
- Exclusão de motorista.
- Cadastro e atualização de entregas.
- Validação de dados obrigatórios.
- IDs inexistentes.
- Dados duplicados.
- Erros de conexão com o banco.
- Respostas corretas da API.
- Segurança e proteção das informações.
-

O arquivo TESTS.md será utilizado para documentar esses cenários.

13 ROADMAP

Etapa 1 — Modelagem

✅ Definir entidades
✅ Criar relacionamentos
✅ Normalizar o banco

Etapa 2 — Arquitetura

✅ Criar estrutura de pastas
✅ Criar rotas
✅ Separar Controllers e Repositories

Etapa 3 — Segurança

✅ Configurar .env
✅ Configurar .gitignore
✅ Padronizar JSON

Etapa 4 — Próxima UC

➡ Implementar o código
➡ Criar testes automatizados
➡ Validar as rotas
➡ Corrigir falhas encontradas

14 RESULTADO ESPERADO

Com a refatoração, a LogiTech Express terá:

- **Banco de dados organizado**

↓

- **Back-End separado por responsabilidades**

↓

- **API padronizada**

↓

- **Maior segurança**

↓

- **Sistema mais escalável e fácil de testar**
-
- O projeto deixa o sistema preparado para a próxima etapa de desenvolvimento e testes.

OBRIGADO PELA ATENÇÃO